Lecture 2-2 — DOM Manipulation: Examples & Explanations

Introduction

This lesson contains three small, clear examples that show how to work with the DOM (Document Object Model) using JavaScript. Each example includes the code followed immediately by a live output window so you can see the effect.

Example 1 — innerText vs innerHTML + building lists

Key idea: **innerText** writes plain text (HTML tags are shown as text). **innerHTML** parses HTML tags and inserts elements. We also show building a list dynamically with JavaScript.

```
<!-- Code (original idea) -->
const txt = "<i>Hello</i>";
document.querySelector("#special").innerText = txt;   // shows literal <i>Hello</i>
document.querySelector("#special2").innerHTML = txt;  // renders italic Hello

const fruits = ["Apple","Orange","Guava"];
let listHTML = "";
fruits.forEach(ele => { listHTML += `<li>${ele}</li>`; });
document.querySelector("#mylist").innerHTML = listHTML;
console.log(document.querySelector("#mylist").innerText);
console.log(document.querySelector("#mylist").innerHTML);
```

Output:

Powered by SPRK Technologies

Explanation (beginner-friendly)

When you assign to **innerText** the browser treats the value as plain text. So if the text contains <i> tags they are shown as characters. When you assign to **innerHTML**, the browser parses the text as HTML and creates elements (like <i> which makes text italic). Building a list by concatenating <li> strings and then setting **innerHTML** is a simple way to render arrays as HTML lists.

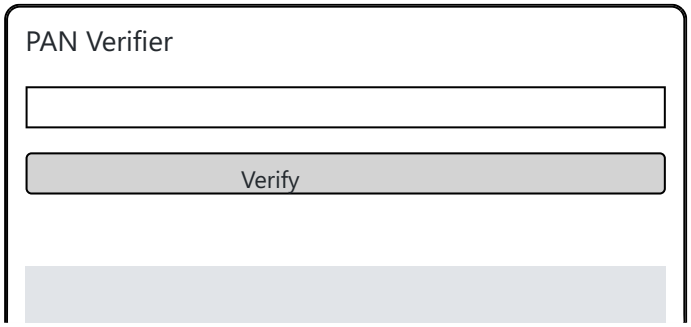
Example 2 — Simple input validation (PAN verifier)

Key idea: read input values with **.value**, use regular expressions to test format, and show error messages by changing text content of an element.

```
<!-- Code (simplified) -->
<input id="inpPAN">
<button onclick="verifyPAN()">Verify</button>
<small class="err"></small>

function panTest(str){
  return /^[A-Z]{5}[0-9]{4}[A-Z]$/.test(str);
}
function verifyPAN(){
  var val = document.querySelector('#inpPAN').value.trim();
  if(val.length === 0){ show error; return; }
  if(!panTest(val)){ show error; return; }
  alert('PAN OK');
}
```

Output:



Powered by SPRK Technologies

Explanation (beginner-friendly)

Inputs (like text boxes) expose their current value via the **.value** property. Regular expressions (the pattern between slashes /.../) let us check whether a string matches a required format. Here the pattern requires 5 uppercase letters, 4 digits, and 1 uppercase letter. When the input is wrong, we set the error element's text so the user sees what went wrong.

Example 3 — Changing list numbering style

Key idea: ordered lists can change their numbering type with the **type** attribute. You can change attributes using **setAttribute** and read them with **getAttribute**.

```
<ol id="myList"><li>Item 1</li>...</ol>
<select onchange="updateListStyle()" id="sel">
  <option value="1">Number</option>
  <option value="i">Roman</option>
  <option value="a">Alphabet</option>
</select>

function updateListStyle(){
  myList.setAttribute('type', sel.value);
}
```

Output:

1. Item 001
2. Item 002
3. Item 003

Powered by SPRK Technologies

Number v

Explanation (beginner-friendly)

HTML attributes (like **type** on an ordered list) control how elements behave or display. JavaScript's **setAttribute** changes that attribute at runtime. This lets users interact with the page to change how it looks.

Summary & Tips

- **Reading vs writing:** use **.value** for form controls and **innerText/innerHTML** for element content.
- **Prefer** innerText when showing user-entered text (avoids injecting HTML).
- **Use** innerHTML when you intentionally want to create elements from an HTML string (be careful with untrusted input).
- **IDs** must be unique on the page — when copying examples on the same page, use different IDs for each live demo.

End of Lecture 2-2

Powered by SPRK Technologies